

VISIÓN POR COMPUTADORA 2 · DETECCIÓN DE OBJETOS

Detección de humo y fuego mediante arquitecturas profundas de detección de objetos

YOLOv8n · YOLO11n · YOLO26s · Faster R-CNN · RT-DETR-L sobre el conjunto de datos D-Fire

Autores: *Marcos Lund, Tomás Mc Nally, Gabriela Sol Salazar*

CEIA - FIUBA

Agosto 2026

Agenda



1. Motivación y objetivo

Por qué detectar humo y fuego automáticamente



2. Dataset D-Fire

Composición, particiones y calidad de las anotaciones



3. Modelos y protocolo

Cinco arquitecturas, tres paradigmas, un mismo protocolo



4. Resultados cuantitativos

Precisión, recall, mAP y velocidad de inferencia



5. Demo

Patrones observados sobre imágenes de test



6. Conclusiones

Qué determina el desempeño y qué sigue

Motivación: detección temprana de incendios



¿POR QUÉ ES IMPORTANTE?



Reduce tiempos de respuesta



Supera la limitación de sensores físicos



Permite **monitoreo remoto** y escalable



¿QUÉ BUSCAMOS?



Obtener un modelo capaz de **detectar fuego y humo** en imágenes.

MODELOS EVALUADOS



YOLOv8n



YOLO11n



YOLO26s



Faster R-CNN



RT-DETR-L



Objetivo: identificar el mejor equilibrio entre desempeño y costo computacional.

El conjunto de datos D-Fire

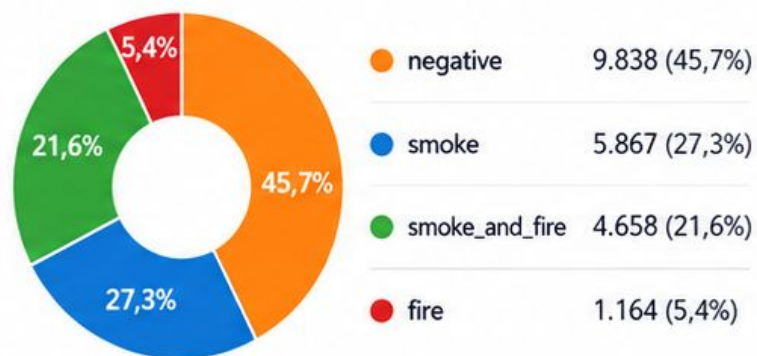
 **21.527** imágenes
total del dataset

 **2** clases
smoke y fire

 **4** tipos de escena
negative, smoke, fire, smoke+fire

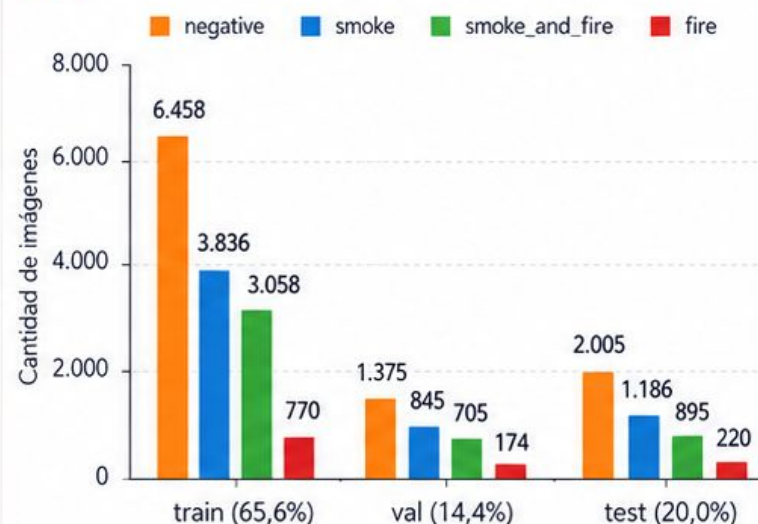
 **Particiones**
train 65,6% · val 14,4% · test 20,0%

Composición del dataset

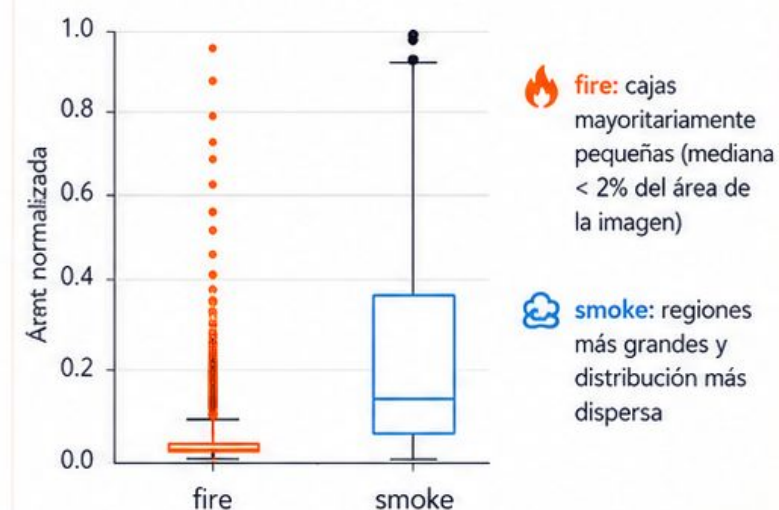


 Las 4 escenas describen el contenido de la imagen; el detector predice solo 2 clases: **smoke** y **fire**.

Distribución por split y tipo de escena



Tamaño de bounding boxes



Hallazgos clave

- ✓ A nivel de imagen hay desbalance aparente: predominan las escenas negativas.
- ✓ A nivel de objeto, el desbalance se atenúa: 14.692 cajas de **fire** vs. 11.865 de **smoke** (≈55% / 45%).
- ✓ No se aplicó rebalanceo de clases.



Calidad de anotación:

8 cajas problemáticas detectadas en test; además se observaron algunas etiquetas intercambiadas y objetos sin anotar en el dataset original.

Muestra del dataset

Diversidad de fuentes (cámaras fijas, fotografías a nivel de suelo), de iluminación y de escala de los fenómenos, sobre las cuatro categorías de escena.



Tres paradigmas, un antecedente directo



Una etapa (YOLO)

Estima clases y cajas directamente en un único paso → alta velocidad de inferencia.



Dos etapas (Faster R-CNN)

Una Region Proposal Network genera regiones candidatas que luego se clasifican y refinan.



Transformer end-to-end (RT-DETR)

Combina las capacidades de los modelos DETR con velocidades adecuadas para tiempo real.



Antecedente directo: Venâncio et al. (videovigilancia para detección temprana de incendios en Brasil, dispositivos de bajos recursos) evaluaron variantes YOLOv4 y construyeron el conjunto de datos D-Fire.

Este trabajo: extiende ese antecedente comparando tres paradigmas arquitectónicos posteriores sobre el mismo dataset, bajo un protocolo experimental común.

Cinco modelos, tres paradigmas



YOLOv8n

Una etapa · nano

Modelo de referencia (baseline) del proyecto.



YOLO11n

Una etapa · nano

Generación posterior; evalúa mejoras arquitectónicas manteniendo bajo costo.



YOLO26s

Una etapa · small

Mayor capacidad moderada, sin recurrir a variantes grandes.



Faster R-CNN

Dos etapas

RPN + refinamiento.
Backbone ResNet-50 con Feature Pyramid Network.



RT-DETR-L

Transformer end-to-end

Detección end-to-end con velocidades adecuadas para tiempo real.

Protocolo experimental

- Mismas particiones de datos, resolución de entrada 640×640 y semilla aleatoria fija para los cinco modelos.
- Todos parten de pesos preentrenados sobre COCO. Los modelos Ultralytics ajustan la red completa; Faster R-CNN ajusta el cabezal y las últimas tres etapas del backbone.
- YOLO: 50 épocas, hiperparámetros por defecto de Ultralytics, early stopping por paciencia.
- Faster R-CNN: receta estándar de torchvision (SGD, momento 0.9, LR inicial 0.005, decaimiento coseno).
- **RT-DETR-L: única desviación deliberada — AdamW con LR 10^{-4} (la selección automática asignaba una configuración SGD inadecuada para un Transformer).**
- Faster R-CNN y RT-DETR-L se entrenan 30 épocas (en lugar de 50) para igualar aproximadamente el presupuesto de cómputo.



Costo computacional

4–18.5h

por corrida en GPU Tesla
T4

1×

búsqueda de hiperparámetros
(sin barrido)

Cada entrenamiento se asocia a un archivo de configuración versionado (arquitectura, pesos iniciales, épocas, tamaños de imagen/batch, LR, optimizador, criterio de early stopping y semilla).

Métricas de evaluación



Precision

$$TP / (TP + FP)$$



Recall

$$TP / (TP + FN)$$



F1-score

$$2 \cdot P \cdot R / (P + R)$$



mAP@0.5

mAP con umbral IoU 0.50



mAP@0.5:0.95

promedio sobre umbrales
0.50–0.95 (métrica principal)



FPS

imágenes por segundo, GPU Tesla
T4

mAP@0.5:0.95 se adopta como métrica principal de comparación porque penaliza las localizaciones imprecisas.

Resultados cuantitativos

Tabla II · Métricas de detección sobre el conjunto de test

Modelo	P	R	F1	mAP@0.5	mAP@0.5:0.95
YOLOv8n	0.766	0.688	0.725	0.759	0.434
YOLO11n	0.752	0.690	0.720	0.753	0.433
YOLO26s	0.786	0.700	0.740	0.773	0.443
Faster R-CNN	0.786	0.726	0.755	0.749	0.403
RT-DETR-L	0.757	0.690	0.722	0.751	0.415

- El rango de mAP@0.5 entre el mejor y el peor modelo es de apenas 2.4 puntos porcentuales (0.749–0.773).
- YOLO26s obtiene los mejores valores de mAP en ambos umbrales y la mejor precisión.
- Faster R-CNN lidera en recall y F1, pero registra el peor mAP en ambos umbrales.

Resultados cuantitativos

smoke



0.788 – 0.841

fire



0.685 – 0.714

mAP@0.5 por clase — rango observado entre los cinco modelos

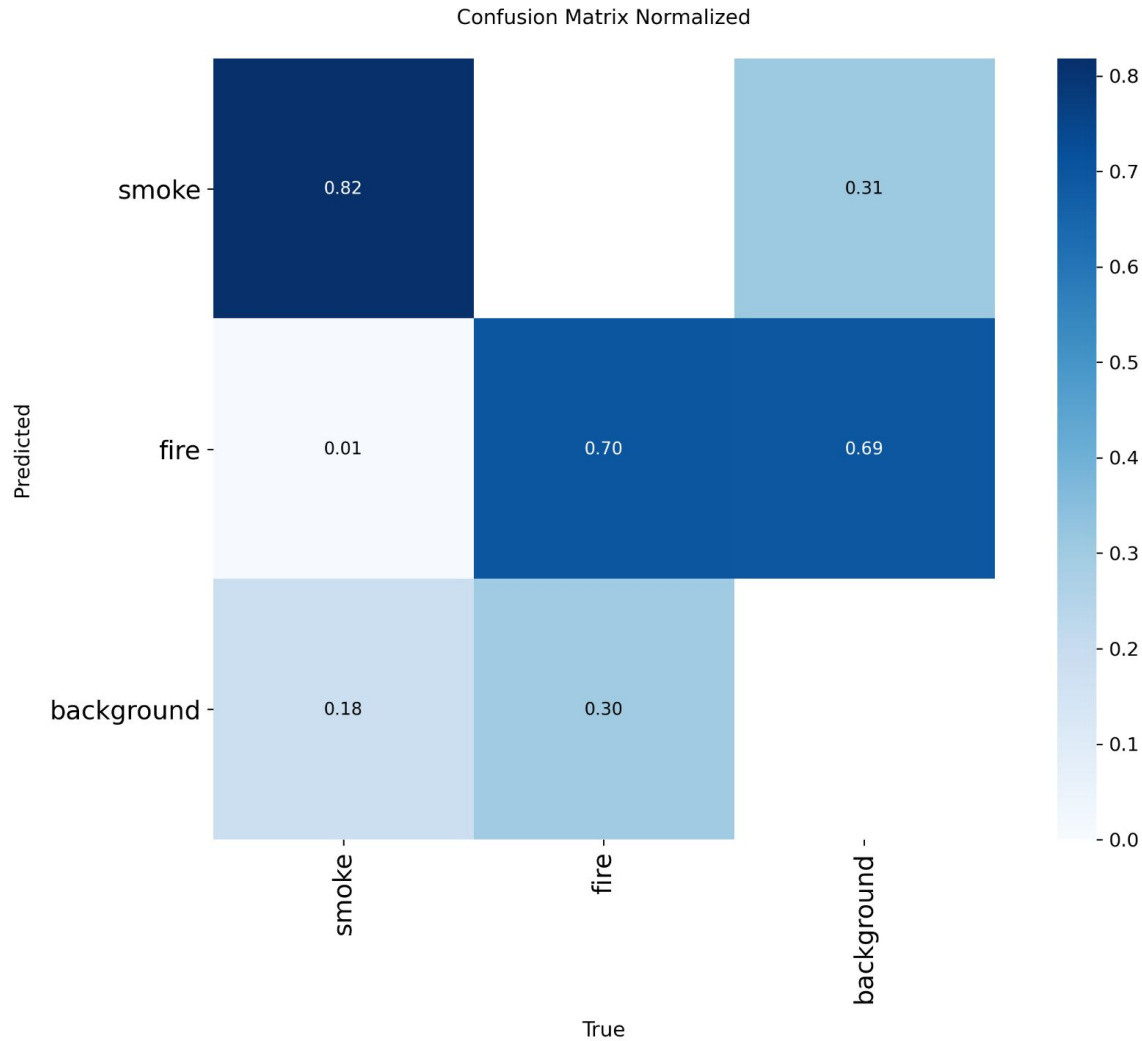
~10 pp

brecha entre clases, en los 5
modelos

*Ninguna arquitectura logra
cerrar esta brecha.*

- El desglose por clase resulta más informativo que las diferencias entre arquitecturas.
- Las cajas de fire son mayoritariamente muy pequeñas.
- La detección de objetos pequeños es una limitación conocida de los detectores a resolución de entrada moderada (640x640).

Resultados cuantitativos



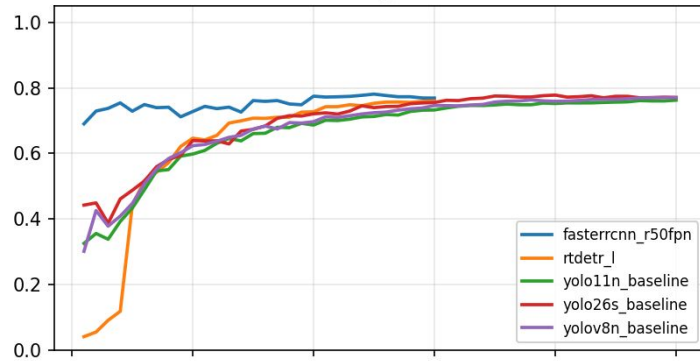
- Confusión entre clases prácticamente nula (~1%).
- El error dominante no es clasificar mal un objeto detectado, sino omitirlo.
- A confianza ≥ 0.25 queda sin detectar el 12–18% de las cajas de humo y el 14–30% de las de fuego

YOLO26s — Matriz de confusión a confianza ≥ 0.25

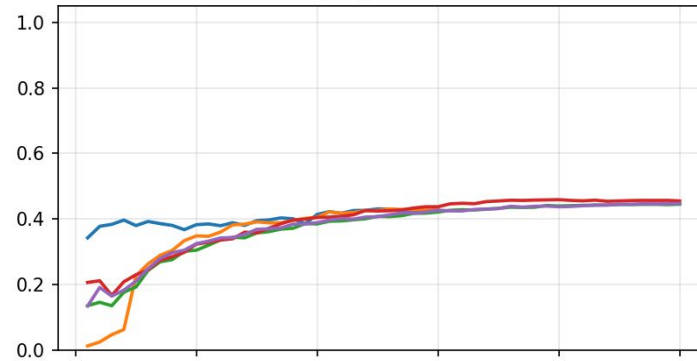
Análisis del entrenamiento

Evolución por época sobre el split de validación

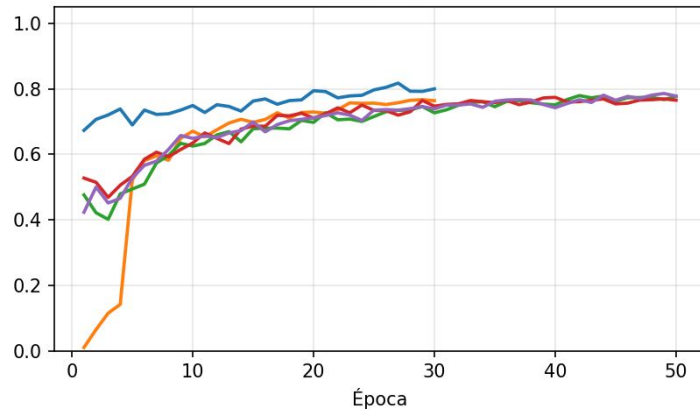
mAP@0.5



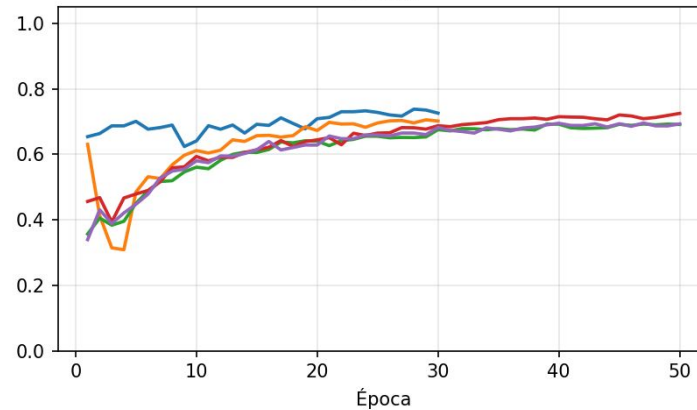
mAP@0.5:0.95



Precisión



Recall

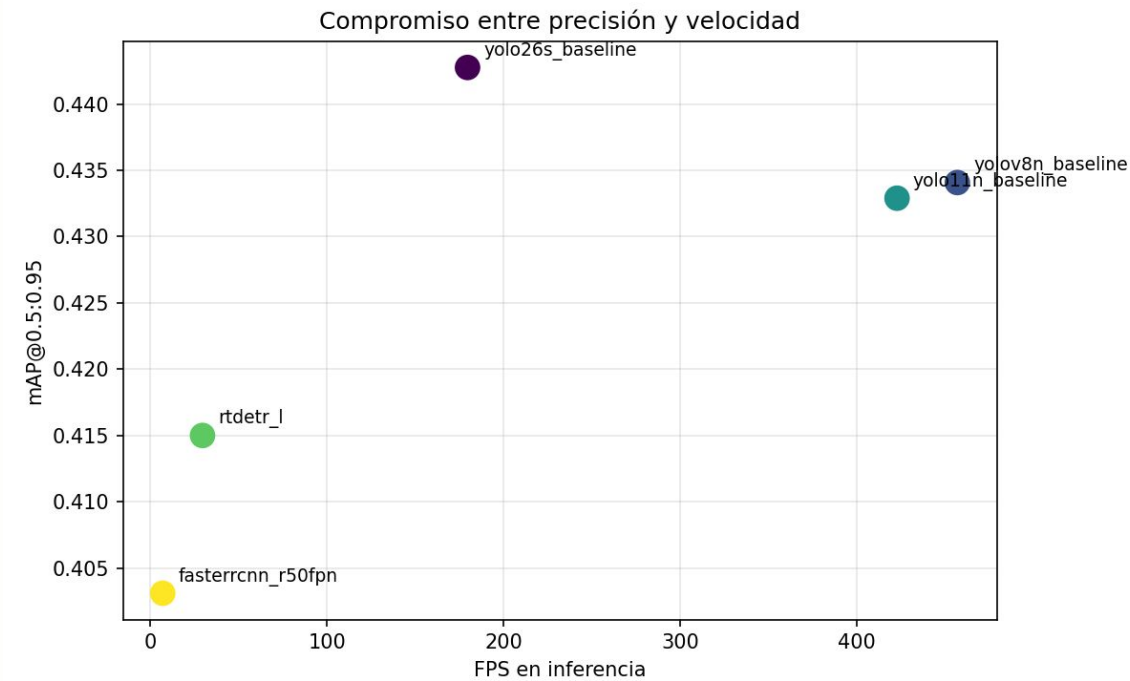


- Faster R-CNN parte alto: mAP@0.5 $\approx$ 0.69 ya en la primera época.
- Las variantes de YOLO comienzan en torno a 0.30-0.45.
- RT-DETR-L parte por debajo de 0.05.
- Hacia el final, las curvas convergen a valores prácticamente indistinguibles
- El presupuesto de épocas fue suficiente para los tres paradigmas.
- YOLO26s alcanzó su mejor punto de control en la época 40 de 50.

Compromiso entre desempeño y velocidad

Tabla III · Eficiencia computacional (GPU Tesla T4)

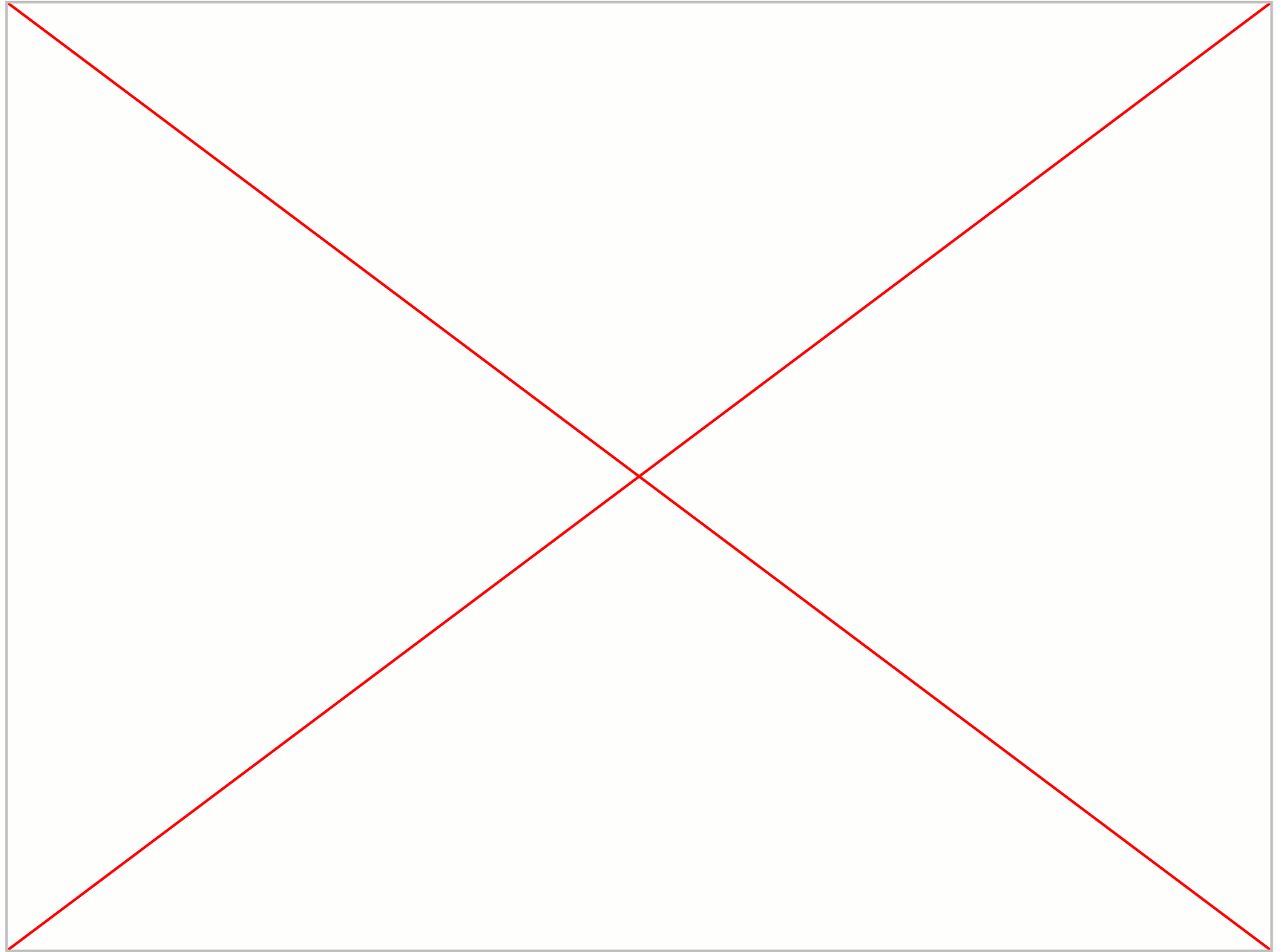
Modelo	Parám. (M)	Épocas	Entren. (min)	FPS
YOLOv8n	3.0	50	243	457.2
YOLO11n	2.6	50	251	423.0
YOLO26s	10.0	50	314	179.7
Faster R-CNN	43.3	30	1112	7.0
RT-DETR-L	32.0	30	563	29.5



- Faster R-CNN solo se justificaría en procesamiento diferido (F1 +3 pp vs. YOLOv8n, pero entrenamiento 18.5h vs. 4h).
- YOLO26s ofrece el punto intermedio más atractivo: mejor mAP de la comparación manteniendo 180 FPS.

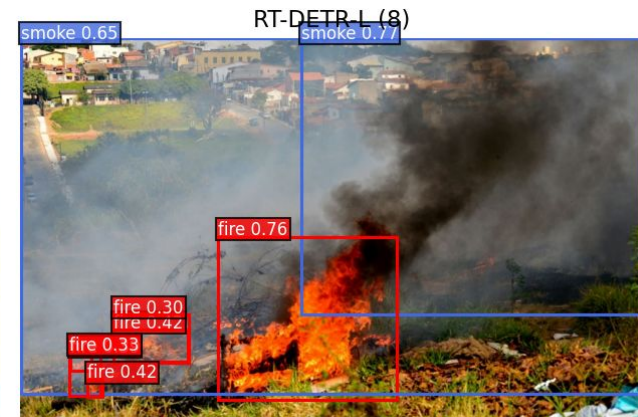
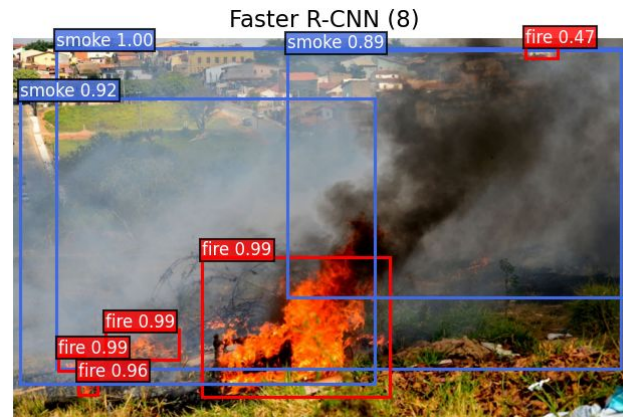
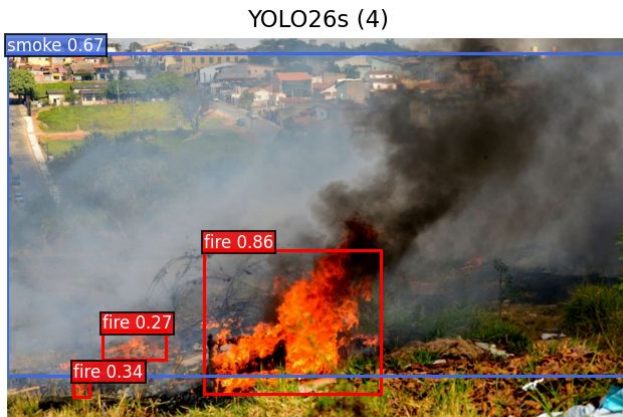
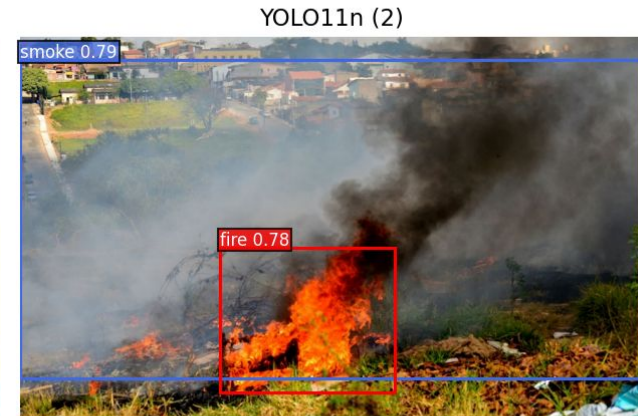
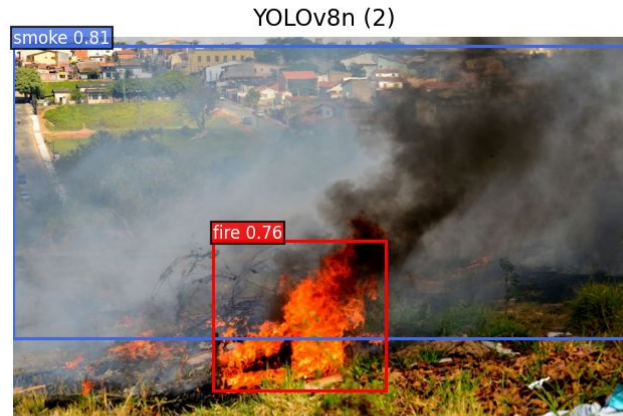
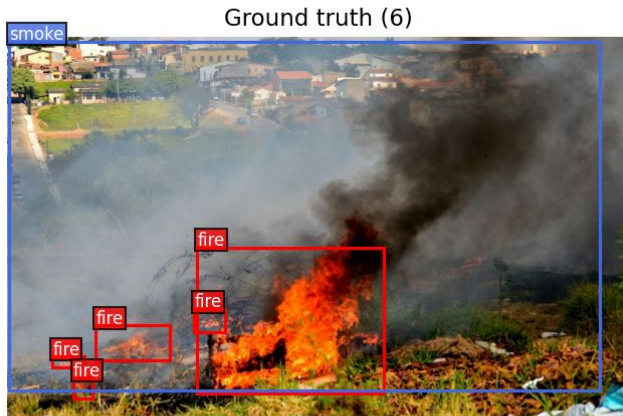
Demo

- Se procesaron imágenes aleatorias del dataset en cada modelo y se compararon las predicciones obtenidas frente al *ground truth*.



Detección de múltiples focos de fuego

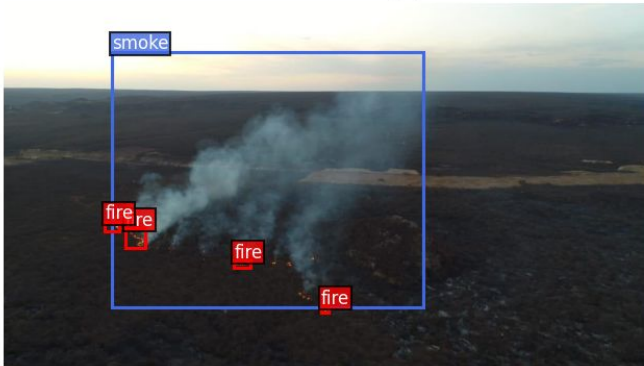
Humo y fuego — test/WEB10538.jpg — conf ≥ 0.25



Detección de múltiples focos de fuego

Humo y fuego — test/WEB11459.jpg — conf ≥ 0.25

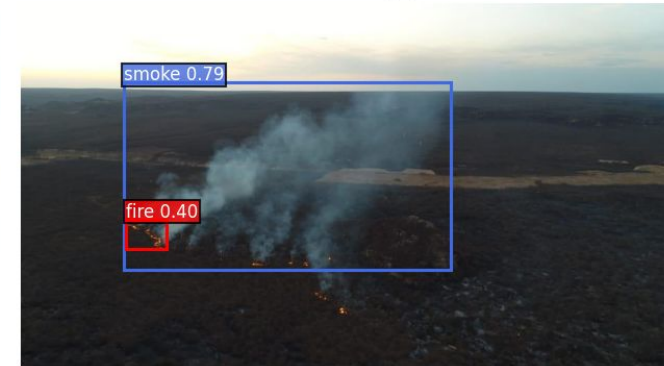
Ground truth (5)



YOLOv8n (1)



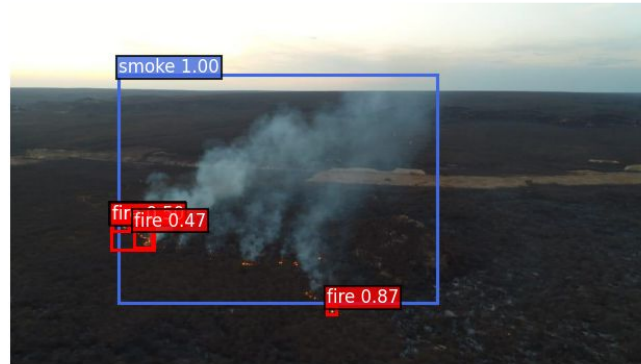
YOLO11n (2)



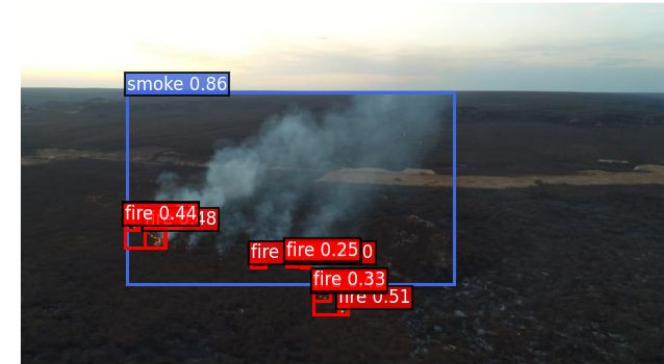
YOLO26s (2)



Faster R-CNN (5)



RT-DETR-L (11)



Detección de múltiples focos de humo

Solo humo — test/WEB11024.jpg — conf ≥ 0.25

Ground truth (2)



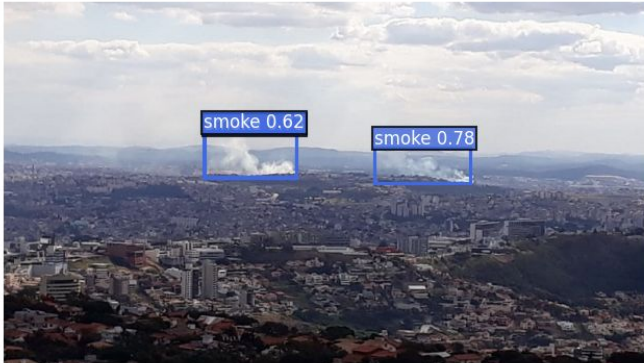
YOLOv8n (2)



YOLO11n (2)



YOLO26s (3)



Faster R-CNN (2)



RT-DETR-L (2)



Detección de objetos pequeños

Objetos pequeños — test/AoF07964.jpg — conf ≥ 0.25

Ground truth (2)



YOLOv8n (0)



YOLO11n (0)



YOLO26s (0)



Faster R-CNN (2)



RT-DETR-L (1)



Análisis de imágenes negativas

Negativa (sin objetos) — test/WEB09664.jpg — conf ≥ 0.25

Ground truth (0)



YOLOv8n (0)



YOLO11n (0)



YOLO26s (0)



Faster R-CNN (0)

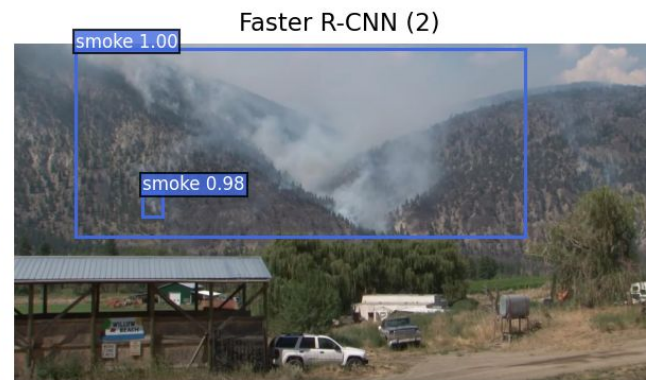
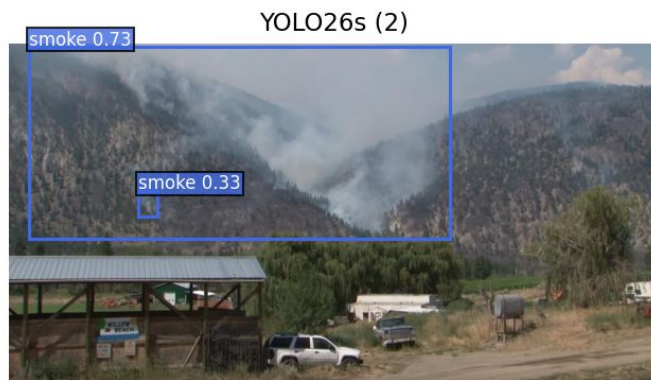


RT-DETR-L (0)



Errores de anotación en el dataset

Solo fuego — test/WEB11741.jpg — conf ≥ 0.25



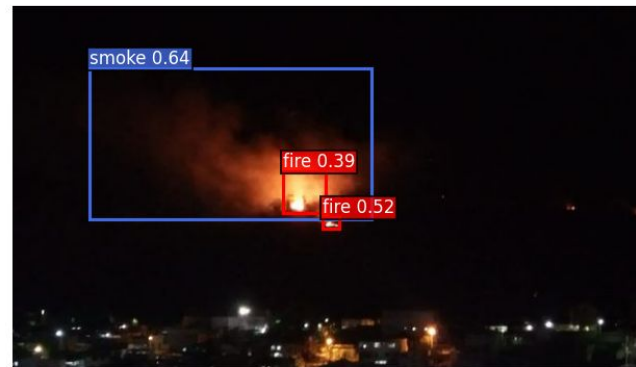
Errores de anotación en el dataset

Solo fuego — test/WEB10446.jpg — conf ≥ 0.25

Ground truth (1)



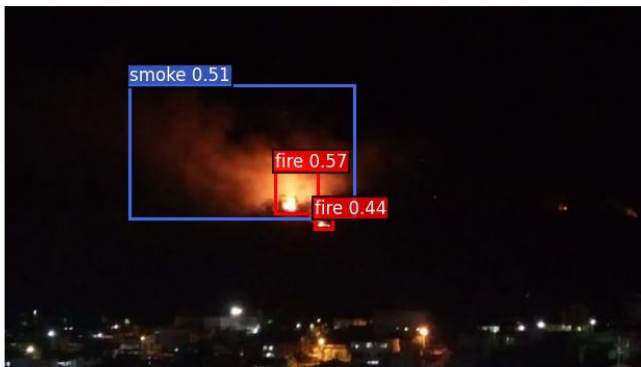
YOLOv8n (3)



YOLO11n (3)



YOLO26s (3)



Faster R-CNN (2)



RT-DETR-L (5)



Conclusiones (I)



1. La arquitectura no es el factor dominante

Cinco modelos muy distintos convergen al mismo techo: el límite lo pone el dataset, no el modelo.



2. Los modelos livianos de una etapa ganan el compromiso

Los YOLO igualan a los pesados en mAP siendo hasta 66× más rápidos: son la opción para tiempo real.



3. El fuego es más difícil que el humo

La brecha se repite en los cinco modelos: las cajas de fuego son pequeñas, y los objetos pequeños son el punto ciego.

Conclusiones (II) y trabajo futuro



4. El error dominante es la omisión, no la confusión

Casi nunca confunden humo con fuego; fallan al no ver el objeto. La prioridad es mejorar el recall.



5. Las métricas son una cota optimista

Secuencias repetidas entre particiones inflan los resultados; los errores de etiquetado penalizan aciertos reales.



Trabajo futuro

Re-particionar por cámara, depurar anotaciones y entrenar con múltiples semillas e hiperparámetros.

Referencias

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, pp. 779–788, 2016.
- [2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems, 2015.
- [3] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in Proc. IEEE/CVF CVPR, pp. 16965–16974, 2024.
- [4] P. V. A. B. de Venâncio, A. C. Lisboa, and A. V. Barbosa, "An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices," Neural Computing and Applications, vol. 34, pp. 15349–15368, 2022.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. <https://github.com/ultralytics/ultralytics>



¡Gracias!

¿Preguntas?

Para conocer más del trabajo: <https://github.com/Gabriela-Sol/VpC2---Deteccion-de-humo-y-fuego>